Procedimiento de Implementación del Software

Sistema de Gestión de Lavado Vehicular (Vehicle Washing)

Planificación, Preparación del Entorno, Despliegue, Carga de Datos Iniciales, Pruebas de Aceptación, Capacitación, Puesta en Marcha y Reversión

Brayan Estiven Patiño Cabrera (líder de proyecto), Juan Camilo Guilombo Osorio (líder de frontend) y Merari Urbano Quintero (líder de backend)

Tecnólogo en Análisis y Desarrollo de Software (ADSO), Servicio Nacional de Aprendizaje (SENA), Centro de Formación: Industria

Ficha 3145556

Instructora: Karol Daniela Correa

27 de octubre de 2026

Tipo de documento: Procedimiento de implementación (equipo de desarrollo, administrador de TI y responsable del negocio) · Versión 2.10

Control de Versiones

La Tabla 1 registra el historial de cambios de este documento. Cada vez que cambie la estrategia de implementación, el entorno de destino o un paso del procedimiento, se debe agregar una fila con la versión, la fecha, la descripción y el responsable.

Tabla 1

Historial de Versiones del Procedimiento de Implementación

Versión	Fecha	Descripción del cambio	Responsable
1.0	2026-10-09	Creación del procedimiento (en español y en inglés) a partir del código de los once repositorios (rama *develop* al 9 de octubre de 2026), de *lavarapido-infra/docker-compose.yml* y de la documentación de despliegue, entornos, operación y calidad del proyecto.	Equipo de desarrollo
2.0	2026-10-16	Revisión contra el código del 16 de octubre de 2026 y el servidor público ya montado. Se actualizaron: servidor público en Azure con Caddy, HTTPS y la web servida con nginx, distribución del APK, primer administrador por variables de entorno, 57 tablas, cola de operations-service, salud de payment-service, pruebas web y de punta a punta en los criterios de entrada, asignación automática en las pruebas de humo, datos del negocio en la carga inicial y riesgos de la implementación.	Equipo de desarrollo
2.1	2026-10-17	Revisión contra el código del 17 de octubre de 2026. Se indicó que la aplicación móvil necesita un APK nuevo por los módulos nativos del informe de inspección (sección 11.7, Fase 6).	Equipo de desarrollo
2.2	2026-10-19	Integración continua con GitHub Actions en los 10 repositorios y despliegue desde GitHub: se actualizaron el alcance (6.2) y el riesgo R-I09.	Equipo de desarrollo
2.3	2026-10-20	Revisión contra el código del 20 de octubre de 2026: criterio E2 con las pruebas del 19 de octubre (655 automatizadas y 19 de punta a punta) y colas de RabbitMQ corregidas (la cola de operations-service no tiene cola de mensajes muertos) y plan de comunicación (el panel no envía avisos generales).	Equipo de desarrollo
2.4	2026-10-21	Revisión contra el código y el servidor del 21 de octubre de 2026: criterio E2 con las pruebas de ese día (689 automatizadas y 19 de punta a punta), respaldo previo automático en el servidor público (fase 3, monitoreo y reversión), prueba de humo H10 (el token de una sesión cerrada deja de servir), variables nuevas con valor por defecto, actualizaciones de la app sin APK nuevo y riesgo R-I11.	Equipo de desarrollo
2.5	2026-10-21	Revisión contra el código de la noche del 21 de octubre de 2026: criterio E2 con la ejecución final de ese día (707 pruebas automatizadas y 19 de punta a punta), variables *FOLLOW_UP_** de operations-service, prueba de humo H11 (lista "Por cerrar" del administrador) y regla de configuración actualizada. Las reservas confirmadas con operario que nadie registró ya no se cierran solas como "No asistió": las resuelve el administrador.	Equipo de desarrollo
2.6	2026-10-23	Revisión contra *develop* del 23 de octubre de 2026 y la versión *v1.0.0*: clonado que funciona (payment-service desde *develop* hasta su primera versión), reversión sin la etiqueta inexistente *v0.9.0* y sin *update-server.sh*, respaldo previo sin *COMPRESSION* (SQL Server Express no la admite), criterio E2 con las pruebas del 23 de octubre (825 automatizadas y 13 de la app móvil; las 19 de punta a punta, solo en local), criterio E4 con la excepción de payment y móvil, variables del compose (ACO-14 cerrada), *INTERNAL_API_KEY* obligatoria para el operario, cuentas demo por variables, aviso de puertos en la red local y despliegue continuo preparado y sin probar (riesgo R-I04). Incluye el contenido de las ramas *fix/A-05*, *fix/A-08*, *fix/B-02* y *fix/E-20* de Merari Urbano.	Equipo de desarrollo
2.7	2026-10-24	Revisión contra *develop* del 24 de octubre de 2026, que corre en el servidor público desde ese día: criterio E2 con las pruebas del 24 de octubre (1 119 automatizadas y 73 de la app móvil, 1 192 en total), criterio E6 cumplido (ninguna P0 ni P1 abierta), clonado con *BRANCH=develop* hasta *v1.1.0*, reversión con *rollback.sh* y las imágenes *:prev* que guarda *update-server.sh*, restauración con *payment-migrate*, la protección que impide bajar de versión, variables nuevas (*AUTH_RATE_LIMIT_**, *TRUSTED_PROXIES*, *ROUTE_*_TIMEOUT*, *START_LATE_LIMIT*, e *INTERNAL_API_KEY* y *BOOKING_SERVICE_URL* en customer-service) y despliegue continuo fusionado en *develop*, a falta de los secretos (R-I04).	Equipo de desarrollo
2.8	2026-10-26	Pago al reservar, desplegado en el servidor público el 26 de octubre de 2026: *INTERNAL_API_KEY* ahora también la exige payment-service, y variables nuevas de booking-service (*BOOKING_CASH_ACTIVE_LIMIT*, *BOOKING_PAYMENT_**, *PAYMENT_SERVICE_URL*) en la Tabla 6.	Equipo de desarrollo
2.9	2026-10-26	R-I04: el despliegue continuo está activo desde el 26 de octubre de 2026 (AC-90).	Equipo de desarrollo
2.10	2026-10-27	Revisión contra *main* del 27 de octubre de 2026: criterio E2 con las pruebas de ese día (1 337, con las 17 de integración de payment en el CI, y 19 de 19 de punta a punta), criterio E4 y clonado con las versiones *v1.1.0*, *v1.1.1* y *v1.1.2* en *main*, protegida en los 10 repositorios, servidor público en *main* por despliegue continuo y verificación de la fase 6 con 58 tablas y las colas actuales de RabbitMQ.	Equipo de desarrollo

Nota. Las fechas se escriben en formato ISO 8601 (año-mes-día). Elaborado por el equipo de desarrollo.

Tabla de Contenido

Control de Versiones 2
4. Introducción 4
4.1 Alcance del Término "Implementación" 4
4.2 Fuentes de Este Documento 4
5. Objetivo 4
5.1 Objetivo General 4
5.2 Objetivos Específicos 4
6. Alcance 5
6.1 Qué Cubre Este Procedimiento 5
6.2 Qué No Cubre Este Procedimiento 5
7. Roles y Responsabilidades 5
8. Estrategia de Implementación 6
8.1 Situación de Partida 6
8.2 Estrategia Seleccionada 6
8.3 Relación con los Procesos de ISO/IEC/IEEE 12207 7
9. Entorno de Destino y Componentes 7
10. Criterios de Entrada 8
11. Procedimiento Paso a Paso 8
11.1 Fase 0: Planificación y Aprobación 8
11.2 Fase 1: Preparación del Entorno 9
11.3 Fase 2: Construcción y Verificación de la Versión 11
11.4 Fase 3: Respaldo Previo 11
11.5 Fase 4: Infraestructura y Base de Datos 12
11.6 Fase 5: Servicios de Backend y Gateway 12
11.7 Fase 6: Aplicaciones Cliente 13
11.8 Fase 7: Configuración Inicial del Negocio 13
11.9 Fase 8: Pruebas de Humo 14
11.10 Fase 9: Pruebas de Aceptación 14
11.11 Fase 10: Capacitación 14
11.12 Fase 11: Puesta en Marcha y Soporte Posterior 15
11.13 Fase 12: Cierre 15
12. Plan de Reversión 16
13. Gestión de la Configuración de la Versión 16
14. Riesgos de la Implementación 18
15. Plan de Comunicación 19
16. Criterios de Salida 19
17. Glosario 19
Referencias 20
18. Anexos 20
Anexo A. Lista de Verificación Previa a la Implementación 21
Anexo B. Casos de Prueba de Aceptación 22
Anexo C. Plantilla de Acta de Aceptación 23
Anexo D. Plantilla de Bitácora de Implementación 24
Anexo E. Resumen de Comandos 25

Nota. La numeración empieza en 4 porque las secciones 1 a 3 son la portada, el control de versiones y la tabla de contenido, que no llevan número en el título. Se conserva así para que sigan valiendo las referencias entre documentos por número de sección (por ejemplo, "Manual de Instalación, sección 10").

4. Introducción

El Sistema de Gestión de Lavado Vehicular (Vehicle Washing), llamado internamente *LavaRapido*, reemplaza el procedimiento manual de un lavadero (agenda en papel, asignación del trabajo a criterio del encargado y pagos sin registro) por una plataforma digital de reservas, asignación de operarios, ejecución y calificación de servicios, verificación de pagos por código QR y un programa de puntos de fidelización. Técnicamente está formado por un gateway, seis microservicios (cinco en Java 21 con Spring Boot 4 y uno en C# con .NET 8), una base de datos SQL Server 2022, un intermediario de mensajes RabbitMQ y dos clientes: una aplicación web en Angular 21 y una aplicación Android en React Native con Expo (Equipo de Desarrollo Vehicle Washing, 2026a).

Este documento define el procedimiento para **implementar** una versión del sistema en su entorno de destino: cómo se planifica, qué se debe cumplir antes de empezar, en qué orden se despliegan los componentes, cómo se cargan los datos iniciales del negocio, cómo se verifica y se acepta el resultado, cómo se capacita a las personas usuarias, cómo se pone en marcha y cómo se revierte si algo falla. Parte de la premisa de que una implementación debe ser repetible, automatizable y verificable (Humble y Farley, 2010) y de que la configuración que cambia entre entornos debe residir en el entorno y no en el código (Wiggins, 2017).

4.1 Alcance del Término "Implementación"

En el contexto del programa ADSO, "implementación del software" designa la **puesta en operación** de una versión en el entorno donde la usarán sus personas usuarias. En la norma ISO/IEC/IEEE 12207:2017, esa actividad corresponde principalmente al proceso de **Transición** (6.4.10), apoyado por los procesos de Integración (6.4.8), Verificación (6.4.9), Validación (6.4.11) y Operación (6.4.12), y por los procesos de gestión técnica de Gestión de la Configuración (6.3.5), Gestión de Riesgos (6.3.4) y Aseguramiento de la Calidad (6.3.8) (International Organization for Standardization [ISO] et al., 2017). La norma usa la palabra *implementation* (6.4.7) para la construcción del elemento de software; para evitar confusiones, este documento llama "construcción" a esa actividad y "implementación" a la puesta en operación. La sección 8.3 relaciona cada fase con su proceso.

4.2 Fuentes de Este Documento

El procedimiento se verificó contra el código de los once repositorios del proyecto (rama *develop* al 21 de octubre de 2026 y, en lo que se actualizó después, *main* del 27 de octubre de 2026), en especial *lavarapido-infra/docker-compose.yml*, *docker-compose.prod.yml*, *.env.example* y la carpeta *deploy*, y contra la documentación de despliegue, entornos, integración continua, operación, calidad y control del proyecto (Equipo de Desarrollo Vehicle Washing, 2026a, 2026b). Los procedimientos detallados de instalación, administración y uso no se repiten aquí: se remite al Manual de Instalación, al Manual de Administrador y al Manual de Usuario, versión 4.0, y al Documento de Despliegue de esta misma carpeta, que registra cómo se montó el servidor público.

5. Objetivo

5.1 Objetivo General

Establecer un procedimiento ordenado, verificable y reversible para poner en operación una versión del Sistema de Gestión de Lavado Vehicular en su entorno de destino, con los datos iniciales del negocio cargados, las personas usuarias capacitadas y la aceptación formal del responsable del negocio.

5.2 Objetivos Específicos

- Definir los roles, las responsabilidades y la estrategia de implementación.
- Fijar los criterios de entrada y de salida que determinan cuándo se puede empezar y cuándo se considera terminada la implementación.
- Describir, por fases, los pasos de preparación, despliegue, configuración inicial, verificación, aceptación, capacitación y puesta en marcha, con su responsable y su evidencia.
- Establecer un plan de reversión que permita volver a la versión anterior sin pérdida de datos.
- Identificar los riesgos propios de la implementación y sus medidas de tratamiento.

6. Alcance

6.1 Qué Cubre Este Procedimiento

- La implementación de una versión etiquetada del backend (seis servicios y gateway), la base de datos, la mensajería y la aplicación web en un equipo con Docker de la red local o en el servidor público (una máquina virtual de Azure con Docker, Caddy y HTTPS que el equipo enciende cuando la necesita). **Aviso:** el *docker-compose.yml* de desarrollo publica en el equipo los puertos 1433 (SQL Server con la cuenta *sa*), 5672 y 15672 (RabbitMQ), 1025 y 8025 (Mailpit) y 3001 a 3006 (cada servicio sin pasar por el gateway); en la red local cualquier equipo los alcanza. Para un uso real se recomienda el servidor público con *docker-compose.prod.yml*, que no publica ninguno; si se usa la red local, bloquear esos puertos en el firewall del equipo y dejar abiertos solo 8080 y 4200.
- La implementación inicial (primera puesta en marcha en el lavadero o en el equipo de evaluación) y las implementaciones posteriores de nuevas versiones.
- La carga de los datos maestros del negocio (horario, bahías, servicios, precios, puntos, cuentas de pago, operarios y promociones).
- Las pruebas de humo, las pruebas de aceptación, la capacitación, la puesta en marcha, el soporte posterior y la reversión.

6.2 Qué No Cubre Este Procedimiento

- Un entorno de producción con alta disponibilidad (varias réplicas, base de datos administrada por el proveedor): la documentación lo declara fuera del alcance académico. La sección 14 lista lo que faltaría para llegar a él. El montaje paso a paso del servidor público está en el Documento de Despliegue.
- La publicación de la aplicación móvil en tiendas de aplicaciones; la aplicación Android se distribuye como APK desde el servidor público (botón y código QR de la página de inicio).
- La configuración de la integración continua: desde el 19 de octubre de 2026 cada repositorio compila y corre sus pruebas con GitHub Actions en cada cambio, y la actualización del servidor también se puede lanzar desde GitHub (Documento de Despliegue). La fase 2 se mantiene para verificar la versión antes de implementarla.
- La migración de datos históricos: el negocio no tiene un sistema anterior; la agenda en papel no se digitaliza (sección 8.1).

7. Roles y Responsabilidades

La Tabla 2 asigna las responsabilidades con la convención RACI: R (responsable de ejecutar), A (aprueba y rinde cuentas), C (consultado) e I (informado).

Tabla 2

Matriz RACI de la Implementación

Actividad	Líder de proyecto (Brayan Estiven Patiño Cabrera)	Líder de backend (Merari Urbano Quintero)	Líder de frontend (Juan Camilo Guilombo Osorio)	Administrador de TI	Responsable del negocio o instructora
Planificación y aprobación de la ventana	A	C	C	C	I
Preparación del entorno	I	C	I	R	—
Construcción y verificación de la versión	A	R	R	I	—
Respaldo previo	I	C	—	R	—
Despliegue de infraestructura, base de datos y servicios	A	R	I	R	—
Despliegue de los clientes	A	I	R	C	—
Configuración inicial del negocio	A	C	C	R	C
Pruebas de humo	A	R	R	R	—
Pruebas de aceptación	R	C	C	C	A
Capacitación	A	R	R	C	I
Puesta en marcha y soporte posterior	A	R	R	R	I
Decisión de reversión	A	R	R	R	I
Cierre y lecciones aprendidas	R	C	C	C	I

Nota. El administrador de TI no es un rol de la aplicación: es quien opera el servidor, Docker y SQL Server. En el equipo académico este papel lo asume la persona designada por el líder de proyecto. Elaborado por los autores.

8. Estrategia de Implementación

8.1 Situación de Partida

Antes del sistema, el lavadero no usaba software: las reservas se anotaban a mano, los turnos se superponían y los pagos en efectivo no dejaban registro del cliente. Por eso **no hay datos heredados que migrar** ni un sistema anterior que deba convivir con el nuevo. Los datos que el sistema necesita para operar son datos maestros que el administrador carga desde el panel (sección 11.8), más los catálogos que las migraciones cargan automáticamente (roles, tipos de vehículo, estados, categorías, tipos de notificación y tres servicios de ejemplo).

8.2 Estrategia Seleccionada

Se adopta una **implementación directa por fases técnicas, con un periodo de piloto controlado**:

1. **Fases técnicas.** Primero la infraestructura y la base de datos, después los servicios y el gateway, después la aplicación web. Cada fase tiene su verificación y no se avanza si falla (sección 11).
2. **Piloto.** Durante la primera semana de operación, el sistema se usa con un grupo reducido de clientes y con todos los operarios, mientras el administrador conserva la agenda en papel como respaldo de contingencia (no como sistema paralelo: la fuente oficial es el sistema).
3. **Corte.** Superado el piloto sin incidentes de severidad P0 o P1 abiertos, se abre el registro a todos los clientes y se retira la agenda en papel.

Se descarta la implementación paralela completa (operar a la vez en papel y en el sistema durante semanas) porque duplica el trabajo del personal sin aportar comparabilidad: el proceso anterior no produce datos que se puedan conciliar con los del sistema. Se descarta la implementación por módulos funcionales porque los flujos dependen unos de otros (no hay pago sin reserva ni asignación sin operario).

8.3 Relación con los Procesos de ISO/IEC/IEEE 12207

Tabla 3

Fases del Procedimiento y Procesos de ISO/IEC/IEEE 12207:2017

Fase	Proceso principal de la norma	Procesos de apoyo
0. Planificación y aprobación	Transición (6.4.10): preparar la transición	Planificación del proyecto (6.3.1), Gestión de decisiones (6.3.3)
1. Preparación del entorno	Transición (6.4.10)	Gestión de la infraestructura (6.2.2)
2. Construcción y verificación de la versión	Integración (6.4.8) y Verificación (6.4.9)	Gestión de la configuración (6.3.5)
3. Respaldo previo	Transición (6.4.10)	Gestión de riesgos (6.3.4)
4 a 6. Despliegue	Transición (6.4.10): ejecutar la transición	Gestión de la configuración (6.3.5)
7. Configuración inicial del negocio	Transición (6.4.10)	Gestión de la información (6.3.6)
8. Pruebas de humo	Verificación (6.4.9)	Aseguramiento de la calidad (6.3.8)
9. Pruebas de aceptación	Validación (6.4.11)	Aseguramiento de la calidad (6.3.8)
10. Capacitación	Transición (6.4.10) y Operación (6.4.12)	Gestión del conocimiento (6.2.6)
11. Puesta en marcha y soporte	Operación (6.4.12)	Mantenimiento (6.4.13)
12. Cierre	Evaluación y control del proyecto (6.3.2)	Gestión del conocimiento (6.2.6)

Nota. Elaborado por los autores a partir de ISO et al. (2017). Los números entre paréntesis corresponden a las cláusulas de la norma.

9. Entorno de Destino y Componentes

El entorno de destino es un equipo o servidor con Docker (Windows 10 u 11 con WSL 2, Ubuntu 22.04 o posterior, o macOS) que ejecuta todos los componentes de la Tabla 4 con *docker compose --profile app*. Los requisitos de hardware y software y los puertos son los de las secciones 6 y 7 del Manual de Instalación (mínimo 4 núcleos, 8 GB de RAM con 5 GB para Docker y 20 GB de disco libre; recomendado 16 GB de RAM y SSD). En el servidor público se agrega *docker-compose.prod.yml*: la web se sirve compilada desde un contenedor con nginx, Caddy publica solo los puertos 80 y 443 con HTTPS y ningún servicio interno queda expuesto. Sin ese archivo (red local) quedan publicados 1433, RabbitMQ, Mailpit y 3001 a 3006: ver el aviso de la sección de alcance.

Tabla 4

Componentes que se Implementan

Componente	Origen	Puerto	Se implementa en
sqlserver	Imagen *mcr.microsoft.com/mssql/server:2022-latest*	1433	Fase 4
db-init	Misma imagen; se ejecuta una vez y crea la base *LavaRapido*	—	Fase 4
payment-migrate	Imagen *liquibase/liquibase:4.29*; aplica los changelogs de pagos	—	Fase 4
rabbitmq	Imagen *rabbitmq:4-management*	5672 y 15672	Fase 4
mailpit (solo pruebas) o SMTP real	Imagen *axllent/mailpit* o Gmail	1025 y 8025	Fase 4
security-service	Repositorio *lavarapido-security-service* (Dockerfile)	3001	Fase 5
customer-service	Repositorio *lavarapido-customer-service*	3002	Fase 5
booking-service	Repositorio *lavarapido-booking-service*	3003	Fase 5
operations-service	Repositorio *lavarapido-operation-service* (nombre en singular)	3004	Fase 5
payment-service (.NET 8)	Repositorio *lavarapido-payment-service* (Dockerfile)	3005	Fase 5
notification-service	Repositorio *lavarapido-notification-service*	3006	Fase 5
api-gateway	Repositorio *lavarapido-api-gateway*	8080	Fase 5
Aplicación web	Repositorio *Front-end-proyecto-web*	4200 en la red local; contenedor con nginx detrás de Caddy en el servidor público	Fase 6
Aplicación móvil	Repositorio *LavadoVehicular-Movil*	APK compilado con EAS y publicado en */app/* del servidor; 8081 (Metro) en desarrollo	Fase 6

Nota. Elaborado por los autores a partir de *docker-compose.yml* y *docker-compose.prod.yml* (16 de octubre de 2026).

10. Criterios de Entrada

La implementación **no comienza** hasta que se cumplan todos los criterios de la Tabla 5. El líder de proyecto verifica cada uno y deja la evidencia en la bitácora (Anexo D).

Tabla 5

Criterios de Entrada

N.°	Criterio	Evidencia
E1	El código de la versión está integrado en *develop* (o en *main*) de los once repositorios, sin trabajo sin confirmar.	*git status* limpio y *git log* con el último commit de cada repositorio.
E2	Las pruebas automáticas pasan: *./mvnw -B verify* en los servicios Java y en el gateway, *dotnet test* en payment-service, *ng test* en la web, *npx jest* en la app móvil y la suite de punta a punta de *lavarapido-infra/e2e* contra los contenedores, que se corre en local porque el CI solo revisa su sintaxis.	Resúmenes sin fallos. CI de *main* del 27 de octubre de 2026: 1 231 pruebas automatizadas en la web, el gateway y los seis servicios (incluidas las 17 de integración de payment, que corren en el CI contra SQL Server 2022 sin omitir ninguna) y 106 de la app móvil, 1 337 en total; la suite de punta a punta local (19 pruebas) pasó 19 de 19 el 27 de octubre de 2026, ya enviando el modo de pago al reservar, y la batería remota (*e2e.yml*, 44 casos) todavía no se ha corrido contra el servidor público.
E3	La aplicación web compila en modo producción (*npx ng build*).	Carpeta *dist/* generada sin errores.
E4	La versión está etiquetada en cada repositorio (por ejemplo, *v1.1.0*) (sección 13). Al 27 de octubre de 2026 los 10 repositorios tienen su versión en *main*: *v1.1.0* en el API Gateway, security-service, operations-service y notification-service; *v1.1.1* en customer-service y booking-service, y *v1.1.2* en la web, la app móvil, payment-service e infra. *main* está protegida en los 10 (solo recibe cambios por *pull request* con una aprobación y el CI en verde) y el servidor público corre *main*, que el despliegue continuo actualiza con cada fusión (Documento de Despliegue, sección 4.3.4).	*git tag* en cada repositorio.
E5	Los manuales de instalación, administrador y usuario corresponden a la versión (versión 4.0 o posterior).	Control de versiones de los manuales.
E6	Las incidencias abiertas de severidad P0 y P1 están resueltas; las P2 y P3 están registradas con su acción en el documento de acciones correctivas, preventivas y de mejoramiento.	Registro de acciones actualizado. Al 24 de octubre de 2026 se cumple: no hay ninguna P0 ni P1 abierta y las 16 abiertas (P2 y P3) tienen acción (documento de Acciones, Tabla 10).
E7	El entorno de destino cumple los requisitos de hardware, software y puertos.	Lista de verificación del Anexo A.
E8	Los secretos de producción están generados y guardados fuera del repositorio (*DB_PASSWORD*, *JWT_SECRET*, *INTERNAL_API_KEY*, *RABBITMQ_PASSWORD* y la contraseña de aplicación del correo).	Confirmación verbal del administrador de TI; nunca copia del valor.
E9	Si el entorno ya tiene datos, existe un respaldo verificado de menos de 24 horas (fase 3).	Archivo *.bak* y salida de RESTORE VERIFYONLY; en el servidor público, las líneas *RESPALDO | OK* y *VERIFICACIÓN | OK* de *~/lavarapido/backups/backup.log*.
E10	La ventana de implementación está aprobada y comunicada (sección 15).	Mensaje de aprobación del líder de proyecto.

Nota. Elaborado por los autores a partir de la definición de terminado y de la estrategia de pruebas del proyecto (Equipo de Desarrollo Vehicle Washing, 2026a).

11. Procedimiento Paso a Paso

Cada fase indica su responsable, su duración estimada, los pasos y la evidencia que se registra en la bitácora (Anexo D). Las duraciones son estimaciones de este documento para una primera implementación en un equipo que cumple los requisitos; las implementaciones posteriores de una nueva versión toman menos tiempo porque las fases 1 y 7 se reducen.

11.1 Fase 0: Planificación y Aprobación

Responsable: líder de proyecto. **Duración estimada:** 1 hora.

1. Definir la versión que se implementa (etiqueta) y el alcance de cambios respecto de la anterior.
2. Elegir la ventana: fuera del horario de atención del lavadero (de lunes a sábado de 7:00 a. m. a 6:30 p. m. en los datos iniciales), con al menos tres horas disponibles.

3. Revisar los riesgos de la sección 14 y confirmar las medidas.
4. Confirmar las personas de cada rol de la Tabla 2 y un canal de comunicación del equipo.
5. Comunicar la ventana a las personas afectadas (sección 15).

Evidencia: registro de la versión, la ventana y los participantes en la bitácora.

11.2 Fase 1: Preparación del Entorno

Responsable: administrador de TI. **Duración estimada:** 1 a 2 horas (primera vez).

1. Instalar y verificar Git, Docker con Compose v2, JDK 21, .NET 8 SDK y Node.js 22 LTS (Manual de Instalación, sección 7.1). En Windows, confirmar que Docker usa el motor Linux con WSL 2.
2. Comprobar que los puertos 1433, 5672, 15672, 1025, 8025, 3001 a 3006, 8080 y 4200 estén libres. Si hay un SQL Server nativo de Windows en el 1433, detener el servicio *MSSQLSERVER*.
3. Clonar los repositorios **uno al lado del otro y con su nombre original**, en la etiqueta de la versión. Al 27 de octubre de 2026 las versiones vigentes son *v1.1.0* (API Gateway, security-service, operations-service y notification-service), *v1.1.1* (customer-service y booking-service) y *v1.1.2* (infra, payment-service y la web); cada una es la punta de *main* y es lo que corre el servidor público, así que clonar con *-b main* da lo mismo mientras no salga otra versión. El repositorio de operaciones se llama *lavarapido-operation-service* (en singular) y *docker-compose.yml* lo construye con ese nombre: clonarlo con otro nombre hace fallar la construcción.

```
mkdir Backend && cd Backend
for repo in lavarapido-security-service lavarapido-api-gateway \
            lavarapido-notification-service lavarapido-operation-service; do
  git clone -b v1.1.0 https://github.com/BrayanCABRERA123/$repo.git
done
git clone -b v1.1.2 https://github.com/BrayanCABRERA123/lavarapido-infra.git
git clone -b v1.1.1 https://github.com/camiloguilombo12/lavarapido-customer-service.git
git clone -b v1.1.1 https://github.com/camiloguilombo12/lavarapido-booking-service.git
git clone -b v1.1.2 https://github.com/merari0322/lavarapido-payment-service.git
cd .. && mkdir Frontend && cd Frontend
git clone -b v1.1.2 https://github.com/BrayanCABRERA123/Front-end-proyecto-web.git
```

1. Crear el archivo de configuración y comprobar que Git lo ignora:

```
cd ../Backend/lavarapido-infra
cp .env.example .env
git check-ignore -v .env      # debe mostrar la regla de .gitignore
```

1. Completar el archivo *.env* con los valores de implementación de la Tabla 6.

Tabla 6

Valores de Configuración para la Implementación

Variable	Valor de implementación	Motivo
DB_PASSWORD	Contraseña fuerte nueva (8 o más caracteres con mayúscula, minúscula, número y símbolo).	SQL Server rechaza las débiles; n reutilizar la de desarrollo.
JWT_SECRET	Valor aleatorio de al menos 32 bytes (*openssl rand -base64 48*).	Firma de los tokens; el mismo par los servicios.

Variable	Valor de implementación	Motivo
INTERNAL_API_KEY	Valor aleatorio (*node -e "console.log(require('crypto').randomBytes(32).toString('base64url'))"*).	Sin ella no se envían por correo lo de reserva, de pago ni los recordat no hay asignación automática y, d *develop* del 23 de octubre de 2026 el operario no puede iniciar ni terr servicios. Desde el 24 de octubre t la recibe customer-service (junto c *BOOKING_SERVICE_URL*, fija e compose) para preguntar a bookin service si un vehículo tiene reserv activas; en producción el compose arranca sin ella. Desde el 26 de oc 2026 también la exige payment-se (*docker-compose.prod.yml*): sin el booking-service no puede compro payment si una reserva digital ven tiene comprobante y no cancela ni por falta de pago.
SPRING_PROFILES_ACTIVE	*prod* (cualquier valor distinto de *dev*).	Con *dev* (valor por defecto del arc Compose) se publica Swagger y s las cuentas de demostración cuya contraseña esté en *DEMO_ADMIN_PASSWORD*, *DEMO_OPERATOR_PASSWORL DEMO_CLIENT_PASSWORD* (va cuenta no se crea).
ALLOWED_EMAIL_DOMAINS	*gmail.com* (por defecto) o la lista de dominios del negocio, separados por comas.	Desde *develop* del 23 de octubre d (A-01), security-service solo acep dominios al registrar, crear usuari cambiar el correo (400 *EMAIL_DOMAIN_NOT_ALLOW* desde el 24 de octubre el inicio de acepta cualquier dominio.
AUTH_RATE_LIMIT_*, TRUSTED_PROXIES, ROUTE_CONNECT_TIMEOUT, ROUTE_READ_TIMEOUT	Opcionales (api-gateway).	Desde el 24 de octubre de 2026: 6 peticiones por minuto por IP en */api/v1/auth/** (*AUTH_RATE_LIMIT_ENABLEL _REQUESTS* y *_WINDOW*: *true*, *1m*), la IP real del cliente solo des privadas (*TRUSTED_PROXIES*) y para conectar y 60 s para leer en c Tienen valor por defecto en el ser todavía no están en el compose ni *.env.example*: para cambiarlas hay agregarlas al bloque *environment* gateway.
START_LATE_LIMIT	Opcional (operations-service).	Desde el 24 de octubre de 2026, h cuánto después de la hora de fin el operario puede iniciar un servicio defecto *PT4H*; después, 409 *SERVICE_TOO_LATE*). Igual que anteriores, no está en el compose.
MESSAGING_ENABLED	*true*	Con *false* (valor por defecto) no s generan notificaciones, ni puntos notificados, ni perfiles por evento.
RABBITMQ_USERNAME, RABBITMQ_PASSWORD	Usuario y contraseña nuevos (no *guest*).	El panel de RabbitMQ no debe qu con credenciales por defecto.
MAIL_ENABLED, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM, MAIL_DOCKER_HOST	*true*, *smtp.gmail.com*, *587*, cuenta del negocio, contraseña de aplicación y *smtp.gmail.com* para Docker.	Mailpit solo atrapa correos; en op real se usa un SMTP real.
CORS_ALLOWED_ORIGINS	Origen real de la aplicación web (por ejemplo, *https://<dominio>* en el servidor público o *http://servidor-lavadero:4200* en la red local).	Los navegadores rechazan otros o
ADMIN_EMAIL, ADMIN_PASSWORD	Correo y contraseña del primer administrador (solo la primera vez).	security-service crea la cuenta al a si todavía no hay ningún administ

Variable	Valor de implementación	Motivo
PUSH_ENABLED	*true* cuando la aplicación móvil instalada es el APK compilado con EAS.	Expo Go no recibe notificaciones remotas.
BOOKING_ZONE	*America/Bogota*	Zona horaria del lavadero; el siste guarda UTC.
SESSION_CHECK_*, BOOKING_EXPIRY_*, FOLLOW_UP_*	Opcionales (comentadas en *.env.example* y *.env.production.example*).	Tienen valor por defecto (chequec sesión del gateway activo con 30 s margen de 60 minutos después del *BOOKING_EXPIRY_GRACE_MI BOOKING_EXPIRY_CHECK_EN* y *BOOKING_EXPIRY_CHECK_IN* ya no tienen efecto desde el 24 de de 2026, porque booking-service c cancelar reservas por su cuenta; seguimiento de operations-service minutos para las reservas que term y nadie cerró en los últimos 28 día *FOLLOW_UP_LOOK_BACK_DA* usa el mismo margen y *INTERNAL_API_KEY*). Son nuev *docker-compose.yml* las pasa a los contenedores.
BOOKING_CASH_ACTIVE_LIMIT, BOOKING_PAYMENT_WINDOW, BOOKING_PAYMENT_EXPIRY_ENABLED, BOOKING_PAYMENT_EXPIRY_INTERVAL	Opcionales (booking-service; comentadas en *.env.example*).	Desde el 26 de octubre de 2026 (p reservar): máximo 1 reserva activa efectivo por cliente, 30 minutos pa el comprobante de una reserva con digital, y la tarea que cancela las v activa y cada 60 segundos (*1*, *30m 60s*). *docker-compose.yml* las pasa contenedor, junto con *PAYMENT_SERVICE_URL* (fija: *http://payment-service:3005*).

Nota. Elaborado por los autores a partir de *.env.example* y *docker-compose.yml*. Los valores secretos se escriben solo en el archivo *.env*; nunca en un mensaje, un documento compartido o un repositorio.

Evidencia: salida de los comandos de versión, lista de repositorios clonados en la etiqueta y confirmación de que *.env* está ignorado.

11.3 Fase 2: Construcción y Verificación de la Versión

Responsables: líder de backend y líder de frontend. **Duración estimada:** 30 a 45 minutos.

1. Ejecutar las pruebas de cada servicio Java y del gateway (*./mvnw -B verify*; en Windows, *.\mvnw.cmd -B verify*).
2. Ejecutar las pruebas de payment-service (*dotnet test* desde *lavarapido-payment-service*).
3. Ejecutar las pruebas de la web (*npx ng test --watch=false*) y construirla (*npm ci* y *npx ng build*).
4. Validar el archivo de Compose y construir las imágenes:

```
cd Backend/lavarapido-infra
docker compose config --quiet && echo "Archivo de Compose válido"
docker compose --profile app build
```

Evidencia: resúmenes de pruebas (número de pruebas y cero fallos), salida de *ng build* y lista de imágenes *lavarapido/*:local* construidas.

11.4 Fase 3: Respaldo Previo

Responsable: administrador de TI. **Duración estimada:** 15 minutos. **Aplica** cuando el entorno ya tiene datos (implementaciones posteriores).

En el servidor público esta fase es automática: *deploy/update-server.sh* corre primero *deploy/backup-db.sh* (respaldo completo con suma de comprobación, *RESTORE VERIFYONLY* y copia comprimida en *~/lavarapido/backups/*, donde se guardan los últimos 14) y, si el respaldo falla, no actualiza (Documento de Despliegue, sección 3.2). Los pasos siguientes son para un equipo de la red local.

1. Generar un respaldo completo con suma de comprobación y verificarlo (Manual de Administrador, sección 14.3):

```
docker compose exec -e SQLCMDPASSWORD="$DB_PASSWORD" sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -b -Q \
  "BACKUP DATABASE [LavaRapido] TO DISK = N'/var/opt/mssql/data/pre_v1.1.0.bak' WITH INIT, CHECKSUM"
docker compose exec -e SQLCMDPASSWORD="$DB_PASSWORD" sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -b -Q \
  "RESTORE VERIFYONLY FROM DISK = N'/var/opt/mssql/data/pre_v1.1.0.bak' WITH CHECKSUM"
docker cp lavarapido-sqlserver:/var/opt/mssql/data/pre_v1.1.0.bak ./backups/
```

El comando no usa *COMPRESSION*: SQL Server Express, la edición del servidor público, no la admite y el respaldo fallaría; *backup-db.sh* comprime después con *gzip*. La contraseña se le entrega a *sqlcmd* en la variable *SQLCMDPASSWORD* y no con *-P*, como hace *backup-db.sh*. En el servidor público no se escribe este comando: se usa *bash ~/lavarapido/lavarapido-infra/deploy/backup-db.sh* y, para probar que el respaldo sirve, *deploy/verify-backup.sh*.

1. Guardar una copia cifrada del archivo *.env* vigente, separada del respaldo de datos.
2. Anotar la etiqueta de la versión que está en operación: es la versión a la que se vuelve si hay que revertir (sección 12).

Evidencia: nombre, tamaño y resultado de la verificación del respaldo (en el servidor público, la línea *RESPALDO | OK* de *backup.log*).

11.5 Fase 4: Infraestructura y Base de Datos

Responsable: administrador de TI con el líder de backend. **Duración estimada:** 10 a 15 minutos.

1. Levantar la infraestructura:

```
cd Backend/lavarapido-infra
docker compose up -d
docker compose ps
```

1. Esperar a que *lavarapido-sqlserver* y *lavarapido-rabbitmq* estén *healthy* y a que *db-init* y *payment-migrate* terminen con código 0.
2. Comprobar que existen los esquemas *payment* y *promotion* (los crea *payment-migrate*). Los demás esquemas los crea cada servicio Java con Liquibase al arrancar en la fase 5.

Criterio para continuar: contenedores sanos y migraciones de pagos con código 0. Si *payment-migrate* falla, no se continúa (sección 12).

Evidencia: salida de *docker compose ps*.

11.6 Fase 5: Servicios de Backend y Gateway

Responsable: líder de backend. **Duración estimada:** 10 a 20 minutos.

1. Levantar los servicios y el gateway:

```
docker compose --profile app up -d
docker compose ps
```

1. Seguir el registro de cada servicio Java hasta ver que Liquibase aplicó sus changelogs y que la aplicación arrancó (*docker compose logs -f security-service*, y así con los demás).
2. Verificar la salud de los siete componentes:

```
for port in 3001 3002 3003 3004 3005 3006 8080; do
  echo "$port: $(curl -s http://localhost:$port/actuator/health)"
done
```

1. Verificar que la base tiene 58 tablas en los nueve esquemas (Manual de Instalación, sección 11.5) y que la topología de RabbitMQ existe: el exchange *carwash.events*, el de mensajes muertos *carwash.events.dlx* y las colas *customer-service.events*, *notification-service.events*, *operations-service.booking-confirmed* y *booking.payment-events*, cada una con su *.dlq*. La cola vieja *operations-service.events* ya no se usa: operations-service la vacía y la borra al arrancar.

Criterio para continuar: las siete respuestas son *UP* y el número de tablas es el esperado. Todos los servicios, incluido payment-service, comprueban la base de datos en su endpoint de salud y responden *DOWN* (503) si no está disponible.

Evidencia: salida del ciclo de salud y del conteo de tablas.

11.7 Fase 6: Aplicaciones Cliente

Responsable: líder de frontend. **Duración estimada:** 15 minutos.

1. **Aplicación web.** En la red local se publica con el servidor de Angular, que reenvía */api* al gateway mediante *proxy.conf.json*:

```
cd Frontend/Front-end-proyecto-web
npm ci
npx ng serve --host 0.0.0.0    # accesible desde otros equipos de la red local
```

En el servidor público la web no usa *ng serve*: el contenedor *web* compila la carpeta *dist/* y la sirve con nginx (con encabezados de seguridad) detrás de Caddy y con HTTPS; *deploy/update-server.sh* lo reconstruye después de cada push (Documento de Despliegue).

1. **Aplicación móvil.** El APK se compila con EAS con *EXPO_PUBLIC_API_URL* apuntando al dominio del servidor y se publica con *npm run publish:apk*, que lo deja en */app/LavaRapidoVehicular.apk* y avisa por push a quienes tienen la aplicación. Para pruebas en la red local se puede usar *npx expo start* con la dirección IPv4 del equipo. La versión del 17 de octubre de 2026 agrega los módulos nativos *expo-image-picker* y *expo-image-manipulator* (fotos del informe de inspección), por lo que exige compilar y publicar un APK nuevo; una actualización por internet (*npm run publish:update*) no basta, porque solo cambia el código de las pantallas. Los cambios de la app del 21 de octubre de 2026 (comprobante comprimido, lupa del QR y pantalla de actualización obligatoria) no agregan módulos nativos y sí llegan con *npm run publish:update*; desde entonces, si el servidor publica un APK con *versionCode* mayor, la app pide instalarlo antes de seguir.

Evidencia: captura de la pantalla de bienvenida servida desde el servidor, con la barra de direcciones visible.

11.8 Fase 7: Configuración Inicial del Negocio

Responsable: administrador de TI con el administrador del negocio. **Duración estimada:** 1 a 2 horas.

Como el perfil *dev* está desactivado, no existen cuentas de demostración. Los datos maestros se cargan en el orden de la Tabla 7, porque cada paso depende de los anteriores.

Tabla 7

Orden de Carga de los Datos Maestros

Paso	Dato	Dónde se carga	Referencia
1	Primera cuenta de administrador	Variables *ADMIN_EMAIL* y *ADMIN_PASSWORD* del archivo de entorno; security-service la crea al arrancar	Manual de Administrador, sección 8.4
2	Horario semanal, descanso y festivos del año	Horarios y bahías > Horario de atención	Manual de Administrador, sección 11.2
3	Bahías de lavado y su estado	Horarios y bahías > Bahías de lavado	Manual de Administrador, sección 11.3
4	Servicios, precios y duración por tipo de vehículo, y puntos de fidelización	Gestión > Servicios	Manual de Administrador, sección 12.2
5	Cuentas de pago con su código QR	Configuración > Medios de pago	Manual de Administrador, sección 11.4
6	Cuentas de operarios y otros administradores	Gestión > Usuarios u Operarios > Nuevo operario	Manual de Administrador, sección 9.3
7	Turno semanal de cada operario	Operarios > Editar disponibilidad	Manual de Administrador, sección 12.1
8	Promociones (cupones de descuento desbloqueables con puntos)	Gestión > Promociones	Manual de Administrador, sección 12.8
9	Verificación de las cuentas de pago	Escanear cada QR desde una cuenta de cliente de prueba y confirmar el titular	Manual de Administrador, sección 11.4
10	Datos del negocio (razón social, NIT, dirección, teléfonos y canales)	Configuración > Datos del negocio	Manual de Administrador, sección 11.1

Nota. Los datos del negocio se guardan en booking-service y son los que ven los clientes en la página de inicio y en el centro de ayuda. Elaborado por los autores.

Evidencia: captura de cada pantalla configurada y lista de cuentas creadas (sin contraseñas).

11.9 Fase 8: Pruebas de Humo

Responsables: líderes de backend y frontend y administrador de TI. **Duración estimada:** 20 minutos.

Las pruebas de humo confirman que la versión funciona de punta a punta antes de entregarla a la aceptación. Se ejecuta la Tabla 8 con cuentas de prueba que se desactivan al terminar.

Tabla 8

Pruebas de Humo

N.°	Prueba	Resultado esperado
H1	Ruta protegida sin token a través del gateway (*curl -i http://localhost:8080/api/v1/bookings/me*).	401 devuelto por booking-service.
H2	Registro de un cliente de prueba e inicio de sesión.	Panel */client*; correo de bienvenida recibido.
H3	Recuperación de contraseña.	Código de 6 dígitos recibido por correo.
H4	Registro de un vehículo y reserva en una hora libre.	Reserva confirmada con bahía y puntos mostrados en el resumen.
H5	Consulta de las cuentas de pago desde la pantalla de pago.	Se listan las cuentas activas con su QR (confirma la conexión de payment-service con la base).
H6	Reserva sin elegir operario y reserva eligiendo uno libre.	El sistema asigna un operario en ambos casos (el elegido en la segunda); notificación al operario.
H7	Inicio y finalización del servicio por el operario.	Reserva COMPLETED; notificaciones al cliente.
H8	Reporte de pago con comprobante de prueba y aprobación por el administrador.	Pago aprobado; correo de pago confirmado y puntos acreditados.
H9	Colas de mensajes muertos.	*notification-service.events.dlq* y *customer-service.events.dlq* en cero.
H10	Cerrar sesión y repetir una llamada con el mismo token (*curl -i -H "Authorization: Bearer \<token\>" https://\<dominio\>/api/v1/bookings/me*).	401: el gateway corta el token de la sesión cerrada (versión del 21 de octubre de 2026 o posterior).
H11	Con el token del administrador, consultar las reservas por cerrar (*GET /api/v1/admin/assignments/to-close*) y abrir el inicio del administrador.	200 con la lista (puede estar vacía); la tarjeta "Por cerrar" solo aparece si hay algo pendiente (versión de la noche del 21 de octubre de 2026 o posterior).

Nota. Elaborado por los autores a partir de la lista de verificación previa a una revisión del proyecto (Equipo de Desarrollo Vehicle Washing, 2026a).

Criterio para continuar: las once pruebas pasan. Una falla en H1 a H8, H10 o H11 activa la evaluación de reversión (sección 12).

11.10 Fase 9: Pruebas de Aceptación

Responsable: líder de proyecto. **Aprueba:** responsable del negocio o instructora. **Duración estimada:** 1 a 2 horas.

1. Ejecutar con el responsable del negocio los casos de aceptación del Anexo B, que cubren los flujos de los tres roles con los datos maestros reales.
2. Registrar el resultado de cada caso (aprobado o no aprobado) y las observaciones.
3. Clasificar cada observación con la severidad de la gestión de incidentes del proyecto (P0 a P3) y registrarla en el documento de acciones correctivas, preventivas y de mejoramiento.
4. Firmar el acta de aceptación (Anexo C). La aceptación puede ser **total** (todos los casos aprobados), **condicionada** (solo observaciones P2 o P3, con fecha de corrección) o **rechazada** (algún caso no aprobado por una falla P0 o P1).

Evidencia: acta de aceptación firmada y resultados de los casos.

11.11 Fase 10: Capacitación

Responsables: líderes de backend y frontend. **Duración estimada:** 3 horas en total.

Tabla 9

Plan de Capacitación por Rol

Público	Contenido	Material	Duración
Administrador del negocio	Panel, reservas, asignación manual de operarios, revisión, aprobación y reembolso de pagos, servicios, precios y puntos, promociones, horarios, bahías, reportes y notificaciones.	Manual de Administrador, secciones 8 a 13	90 minutos
Administrador de TI	Arranque y parada, registros, respaldo y restauración, mantenimiento preventivo, incidentes y reversión.	Manual de Administrador, secciones 14 a 17; este procedimiento, sección 12	45 minutos
Operarios	Agenda, servicios asignados, iniciar y terminar, notificaciones y calificaciones.	Manual de Usuario, sección 10.2	30 minutos
Personal de atención al cliente	Registro, reserva, pago con QR, cupones y puntos, historial y calificación, para orientar a los clientes.	Manual de Usuario, sección 10.1	30 minutos

Nota. Cada sesión termina con un ejercicio práctico en el entorno implementado con cuentas de prueba. Elaborado por los autores.

Evidencia: lista de asistencia con fecha, tema y firma.

11.12 Fase 11: Puesta en Marcha y Soporte Posterior

Responsables: todo el equipo. **Duración:** piloto de una semana y soporte reforzado de dos semanas.

1. Desactivar las cuentas de prueba creadas en las fases 8 y 9.
2. Habilitar el sistema para el grupo piloto y luego para todos los clientes (sección 8.2).
3. Durante las dos primeras semanas, el administrador de TI ejecuta a diario las tareas de la Tabla 10 y el equipo atiende las incidencias con los tiempos de la gestión de incidentes del proyecto (P0 inmediato; P1 el mismo día).

Tabla 10

Monitoreo Diario Durante el Soporte Posterior

Tarea	Cómo se verifica
Salud de los siete componentes	Ciclo de la sección 11.6.
Errores de las últimas 24 horas	*docker compose logs --since 24h | grep -i error*
Colas de mensajes muertos	Panel de RabbitMQ (*http://localhost:15672*).
Pagos por revisar y reservas sin operario	Panel del administrador.
Respaldo diario generado y verificado	Archivo del día y bitácora de respaldos; en el servidor público, *tail -5 ~/lavarapido/backups/backup.log* con *RESPALDO | OK* y *VERIFICACIÓN | OK* del día, y *grep FALLÓ* sin resultados.
Estado de los contenedores (servidor público)	*docker compose ps*: todos *healthy*.
Espacio en disco	*docker system df*; uso por debajo del 80 %.

Nota. Adaptado del plan de mantenimiento preventivo del Manual de Administrador, sección 17.6.

Evidencia: registro diario del monitoreo y de las incidencias atendidas.

11.13 Fase 12: Cierre

Responsable: líder de proyecto. **Duración estimada:** 1 hora.

1. Confirmar los criterios de salida (sección 16).
2. Realizar una retrospectiva de la implementación y registrar las lecciones aprendidas en el documento de acciones correctivas, preventivas y de mejoramiento.
3. Actualizar el registro de riesgos, el backlog técnico y los manuales si la implementación reveló diferencias.
4. Archivar la bitácora, el acta de aceptación y las listas de asistencia con la versión implementada.

12. Plan de Reversión

La reversión devuelve el entorno a la versión anterior cuando la nueva no puede operar. Se decide con los criterios de la Tabla 11 y la ejecuta el líder de backend con el administrador de TI.

Tabla 11

Criterios de Reversión

Situación	Decisión
Falla *payment-migrate* o una migración Liquibase de un servicio Java	Revertir de inmediato.
Una prueba de humo H1 a H8, H10 o H11 falla y no se corrige en 30 minutos	Revertir.
Incidente P0 en las primeras 24 horas de operación	Revertir.
Incidente P1 sin corrección ni alternativa en el mismo día	Evaluar con el líder de proyecto; revertir si afecta reservas o pagos.
Observaciones P2 o P3	No revertir; corregir en una versión posterior.

Nota. Elaborado por los autores a partir de la clasificación de severidad del proyecto.

Pasos de reversión:

1. Comunicar la decisión y detener el acceso de las personas usuarias.
2. Detener los servicios y el gateway, dejando SQL Server en ejecución:

```
docker compose stop api-gateway security-service customer-service booking-service \
  operations-service payment-service notification-service
```

3. Restaurar el respaldo previo de la fase 3 (Manual de Administrador, sección 15.2). En el servidor público se usa *bash deploy/restore-db.sh \<respaldo.bak.gz\>*: pide escribir *RESTAURAR*, guarda antes un respaldo de cómo estaba la base, detiene los servicios, restaura y los vuelve a levantar. Los changesets de Liquibase solo avanzan, por lo que **no se deshace una migración con otra**: se restaura la base de datos al estado anterior a la versión.
4. En cada repositorio afectado, volver a la versión anterior anotada en la fase 3: su etiqueta (la primera que existe es *v1.0.0*; no hay versiones anteriores etiquetadas) o, si no tiene, su commit (*git checkout \<etiqueta o commit\>*); luego reconstruir y levantar: *docker compose --profile app up -d --build*. En el servidor público no se hace así, porque *update-server.sh* vuelve a bajar la punta de la rama: se revierte el cambio en GitHub (*git revert* en una rama con su PR) y se despliega de nuevo o, de urgencia, se vuelve a las imágenes que *update-server.sh* guardó como *:prev* con *bash deploy/rollback.sh* (todos los servicios) o *rollback.sh \<servicio\>*, que pide escribir *VOLVER*, recrea los contenedores sin compilar y espera a que queden sanos (Documento de Despliegue, sección 3.4). Desde el 24 de octubre de 2026 *restore-db.sh* corre *payment-migrate* al final y *setup-server.sh* nunca baja de versión sin *ALLOW_DOWNGRADE=1*.
5. Restaurar el archivo *.env* anterior si la versión había agregado variables.
6. Repetir las pruebas de humo de la sección 11.9.
7. Comunicar el restablecimiento indicando la fecha y la hora de los últimos datos recuperados; los pagos recibidos después del respaldo se concilian con el extracto bancario.
8. Registrar la reversión como no conformidad en el documento de acciones correctivas, preventivas y de mejoramiento, con su análisis de causa raíz.

Tiempo objetivo de reversión: 1 hora desde la decisión. **Pérdida máxima de datos:** los registros posteriores al respaldo de la fase 3.

13. Gestión de la Configuración de la Versión

- **Etiquetado.** Cada versión implementada se etiqueta en los repositorios de código con el mismo nombre (versionado semántico: *vMAYOR.MENOR.PARCHE*, por ejemplo, *v1.0.0*). Las etiquetas permiten reconstruir exactamente lo implementado y revertir. Estado al 24 de octubre de 2026: *v1.0.0* en 9 repositorios (web, api-gateway, security, customer, booking, operations, notification e infra desde

el 23 de octubre, y payment-service desde el 24, con su PR #2 fusionado); falta la app móvil (PR #1 abierto), y *develop* ya tiene los arreglos del 23 y del 24 de octubre para *v1.1.0*.

- **Ramas.** Se implementa desde *main*, que tiene la versión etiquetada y está protegida en los 10 repositorios; nunca desde una rama *feature/**. Desde el 26 de octubre de 2026 el servidor público corre *main*, y el despliegue continuo lo actualiza con cada fusión aprobada a *main* (Documento de Despliegue, sección 4.3.4).
- **Imágenes.** El archivo de Compose construye imágenes *lavarapido/\<servicio\>:local*. Para conservar la versión anterior, antes de reconstruir se etiqueta cada imagen (*docker tag lavarapido/booking-service:local lavarapido/booking-service:prev*, o con el nombre de la versión vigente, *:v1.0.0*); para volver se hace la operación inversa y *docker compose up -d --no-build \<servicio\>*. En el servidor público lo hacen solos desde el 24 de octubre de 2026: *update-server.sh* etiqueta *:prev* antes de reconstruir y *rollback.sh* vuelve a esa imagen.
- **Base de datos.** El esquema se versiona con Liquibase; cada servicio guarda su historial en *DATABASECHANGELOG_\<SERVICIO\>* (pagos en *DATABASECHANGELOG_PAYMENT*). Un changeset ejecutado nunca se edita.
- **Configuración.** *.env.example* se versiona; *.env* no. Toda variable nueva se agrega a *.env.example* con un comentario y al bloque *environment* del servicio en *docker-compose.yml*. Las del 21 de octubre de 2026 (*SESSION_CHECK_**, *BOOKING_EXPIRY_** y *FOLLOW_UP_**, nueve variables) ya cumplen esta regla (acción ACO-14, cerrada el 23 de octubre de 2026), igual que *ALLOWED_EMAIL_DOMAINS* y *DEMO_*_PASSWORD* del 23 de octubre. Las del 24 de octubre en el gateway (*AUTH_RATE_LIMIT_**, *TRUSTED_PROXIES*, *ROUTE_*_TIMEOUT*) y en operations-service (*START_LATE_LIMIT*) tienen valor por defecto pero todavía no están en *.env.example* ni en el compose (ACO-40).
- **Documentación.** El control de versiones de cada manual registra la versión del software a la que corresponde.

14. Riesgos de la Implementación

Tabla 12

Riesgos Específicos de la Implementación y su Tratamiento

ID	Riesgo	Probabilidad	Impacto	Tratamiento
R-I01	El repositorio de operaciones se clona como *lavarapido-operations-service* y la construcción de Compose falla.	Media	Alto	Clonar con el nombre original *lavarapido-operation-service* (fase 1).
R-I02	Quedan *MESSAGING_ENABLED=false* o *INTERNAL_API_KEY* vacía y no llegan notificaciones ni correos.	Media	Medio	Valores de la Tabla 6 y pruebas H2, H7 y H8.
R-I03	Se implementa con el perfil *dev* y quedan cuentas de demostración y Swagger expuestos.	Media	Alto	*SPRING_PROFILES_ACTIVE=prod*; Anexo A.
R-I04	El servidor público está apagado cuando se necesita (se enciende bajo demanda).	Media	Medio	Avisar con anticipación al responsable de DevOps y ejecutar *deploy/update-server.sh* después del último push. Con el despliegue continuo (activo desde el 26 de octubre de 2026, AC-90), lo que entra a *main* con la máquina apagada se despliega solo al encenderla (*deploy/boot-update.sh*, instalado en el servidor ese día); para la exposición la máquina se enciende antes y se congela *main*.
R-I05	No hay límite de solicitudes en el gateway ni en Caddy (TD-022); el bloqueo de cuentas solo protege cada cuenta por separado.	Media	Medio	Contraseñas largas y únicas para los administradores; revisar los registros de Caddy.
R-I06	Un código QR o una cuenta de pago mal cargada envía el dinero a un tercero.	Baja	Alto	Paso 9 de la Tabla 7.
R-I07	Pérdida de datos por *docker compose down -v* o por un volumen dañado.	Baja	Alto	Respaldo previo (fase 3) y respaldo diario, ambos verificados; en el servidor público son automáticos (Documento de Despliegue, sección 3.2).
R-I08	En la red local la web se sirve con *ng serve* y sin HTTPS.	Media	Medio	Usarla solo dentro de la red del lavadero; fuera de ella, usar el servidor público con HTTPS.
R-I09	La integración continua no corre las pruebas de punta a punta, que necesitan el sistema completo, y una regresión entre servicios puede pasar sin detectarse.	Baja	Medio	Confirmar que GitHub Actions está en verde en los 10 repositorios y ejecutar la suite de punta a punta y las pruebas de humo antes de cada despliegue.
R-I10	Los permisos por rol editables en el panel se confunden con control de acceso real (ADR-015).	Baja	Medio	Explicarlo en la capacitación; la autorización real la da el rol del token, y desde el 21 de octubre de 2026 la pestaña de roles lo dice.
R-I11	Si security-service no responde, el gateway deja pasar las peticiones sin confirmar la sesión, y un token de una sesión cerrada sirve hasta que vence (máximo 1 hora); con security-service activo, hasta 30 segundos.	Baja	Medio	Revisar la salud de security-service (fase 5 y Tabla 10) y el registro del gateway (*No se pudo confirmar la sesión*).

Nota. Elaborado por los autores a partir del registro de riesgos, del backlog técnico y de la verificación del código (Equipo de Desarrollo Vehicle Washing, 2026a).

Para llevar el sistema a un entorno de producción real, además de lo anterior faltaría: SQL Server y RabbitMQ alojados con los secretos en un almacén del proveedor, un inicio de sesión de base de datos por servicio en lugar de *sa*, un límite de solicitudes y un Outbox para los eventos si no se acepta perder un evento durante una caída del intermediario (Equipo de Desarrollo Vehicle Washing, 2026a).

15. Plan de Comunicación

Tabla 13

Plan de Comunicación de la Implementación

Momento	Mensaje	Destinatarios	Canal	Responsable
48 horas antes	Versión, ventana, impacto y contacto.	Equipo, responsable del negocio y operarios	Chat del equipo y aviso en el lavadero	Líder de proyecto
Inicio de la ventana	Inicio de la implementación.	Equipo	Chat del equipo	Líder de proyecto
Fin de cada fase	Resultado y evidencia.	Equipo	Chat del equipo y bitácora	Responsable de la fase
Decisión de reversión	Motivo y tiempo estimado.	Equipo y responsable del negocio	Chat del equipo	Líder de proyecto
Puesta en marcha	Sistema disponible, dirección de acceso y canal de soporte.	Personal del lavadero y clientes del piloto	Mensaje directo y aviso en el lavadero (la web y la app no tienen una pantalla para enviar avisos generales; acción de mejora AM-13)	Administrador del negocio
Cierre	Resultado, lecciones aprendidas y pendientes.	Equipo, responsable del negocio e instructora	Reunión de cierre	Líder de proyecto

Nota. Elaborado por los autores a partir de la gestión de incidentes del proyecto.

16. Criterios de Salida

La implementación se considera **terminada** cuando se cumplen todos los criterios siguientes:

- Las pruebas de humo de la Tabla 8 pasan en el entorno implementado.
- El acta de aceptación está firmada con aceptación total o condicionada (Anexo C).
- Los datos maestros de la Tabla 7 están cargados y verificados.
- Las personas de cada rol recibieron la capacitación (Tabla 9).
- Existe un respaldo verificado posterior a la configuración inicial y el respaldo diario está programado.
- El periodo de soporte posterior terminó sin incidentes P0 o P1 abiertos.
- La bitácora, el acta y las lecciones aprendidas están archivadas con la versión.

17. Glosario

Tabla 14

Glosario de Términos de la Implementación

Término	Definición
Acta de aceptación	Documento firmado en el que el responsable del negocio acepta, condiciona o rechaza la versión implementada.
Bitácora de implementación	Registro cronológico de los pasos ejecutados, sus responsables y sus evidencias.
Criterio de entrada	Condición que debe cumplirse antes de iniciar la implementación.
Criterio de salida	Condición que debe cumplirse para dar por terminada la implementación.
Datos maestros	Datos de configuración del negocio que el sistema necesita para operar (horario, bahías, servicios, precios y cuentas de pago).
Etiqueta (tag)	Nombre fijo que Git asigna a un commit para identificar una versión.
Perfil de Compose	Grupo de servicios que solo se inicia cuando se solicita (*--profile app*).
Piloto	Periodo inicial de operación con un grupo reducido de personas usuarias.
Prueba de aceptación	Prueba que realiza el responsable del negocio para confirmar que el sistema satisface sus necesidades (validación).
Prueba de humo	Prueba rápida que confirma que las funciones principales operan después de un despliegue (verificación).
RACI	Matriz que indica quién es responsable, quién aprueba, a quién se consulta y a quién se informa en cada actividad.
Reversión	Retorno controlado a la versión anterior del software y de los datos.
Transición	Proceso de ISO/IEC/IEEE 12207 que establece la capacidad de un sistema para operar en su entorno de destino.
Ventana de implementación	Periodo acordado, fuera del horario de atención, en el que se ejecuta la implementación.

Nota. Elaborado por los autores a partir de ISO et al. (2017) y de la documentación del proyecto.

Referencias

Equipo de Desarrollo Vehicle Washing. (2026a). *Vehicle washing management system: Project documentation* (vehicle-w-docs) [Documentación de proyecto no publicada]. GitHub. https://github.com/code-sena/vehicle-w-docs

Equipo de Desarrollo Vehicle Washing. (2026b). *lavarapido-infra* (rama develop) [Software]. GitHub. https://github.com/BrayanCABRERA123/lavarapido-infra

Humble, J., y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Organization for Standardization, International Electrotechnical Commission e Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). https://www.iso.org/standard/63712.html

Servicio Nacional de Aprendizaje. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador* [Material de formación no publicado]. Programa Tecnólogo en Análisis y Desarrollo de Software.

Wiggins, A. (2017). *The twelve-factor app: III. Config*. https://12factor.net/config

18. Anexos

Anexo A. Lista de Verificación Previa a la Implementación

Tabla 15

Lista de Verificación Previa

Control	Cumple
Criterios de entrada E1 a E10 cumplidos (Tabla 5)	Sí / No
Repositorios clonados en la etiqueta, uno al lado del otro y con su nombre original (incluido *lavarapido-operation-service*)	Sí / No
.env creado, ignorado por Git y con los valores de la Tabla 6	Sí / No
SPRING_PROFILES_ACTIVE distinto de *dev*	Sí / No
MESSAGING_ENABLED=true e *INTERNAL_API_KEY* definida	Sí / No
Credenciales de RabbitMQ distintas de *guest*	Sí / No
SMTP real configurado y probado	Sí / No
Puertos libres y SQL Server nativo detenido	Sí / No
Respaldo previo verificado (si el entorno tenía datos)	Sí / No
Solo el gateway y la aplicación web accesibles desde fuera del servidor	Sí / No

Nota. Elaborado por los autores.

Anexo B. Casos de Prueba de Aceptación

Tabla 16

Casos de Prueba de Aceptación

ID	Rol	Caso	Resultado esperado	Resultado
CA-01	Cliente	Registrarse con datos válidos y aceptar los documentos legales.	Cuenta creada y correo de bienvenida.	Aprobado / No aprobado
CA-02	Cliente	Intentar registrarse con un correo que no termina en @gmail.com.	Mensaje de validación; no se crea la cuenta.	Aprobado / No aprobado
CA-03	Cliente	Registrar un vehículo con placa válida y otro con placa inválida.	El primero se guarda; el segundo se rechaza con su mensaje.	Aprobado / No aprobado
CA-04	Cliente	Reservar un servicio en una hora libre.	Reserva confirmada con bahía, total y puntos que suma.	Aprobado / No aprobado
CA-05	Cliente	Intentar reservar una hora que otra persona acaba de tomar.	Mensaje de hora tomada y hasta cinco alternativas.	Aprobado / No aprobado
CA-06	Cliente	Cancelar una reserva antes de su hora.	Reserva cancelada y aviso por correo.	Aprobado / No aprobado
CA-07	Administrador	Asignar un operario disponible y ver el motivo de bloqueo de uno no disponible.	Asignación confirmada; motivo visible para el no disponible.	Aprobado / No aprobado
CA-08	Operario	Iniciar y terminar el servicio asignado el día de la reserva.	Estados En progreso y Completado; avisos al cliente.	Aprobado / No aprobado
CA-09	Cliente	Pagar con QR, adjuntar comprobante y referencia.	Pago en revisión.	Aprobado / No aprobado
CA-10	Administrador	Aprobar el pago tras confirmar el dinero en el banco.	Pago aprobado; correo al cliente; puntos acreditados.	Aprobado / No aprobado
CA-11	Administrador	Rechazar un pago con motivo.	Pago rechazado; correo al cliente con el motivo.	Aprobado / No aprobado
CA-12	Cliente	Canjear un cupón desbloqueado al pagar.	Descuento aplicado al total y correo de cupón canjeado.	Aprobado / No aprobado
CA-13	Cliente	Calificar un servicio completado.	Calificación visible para el operario; no se permite calificar dos veces.	Aprobado / No aprobado
CA-14	Administrador	Registrar un festivo y comprobar que el día no ofrece horas.	El cliente ve "El lavadero está cerrado ese día".	Aprobado / No aprobado
CA-15	Administrador	Exportar el reporte del mes a PDF y Excel.	Archivos descargados con los datos del periodo.	Aprobado / No aprobado
CA-16	Todos	Cambiar el idioma y el tema, cerrar sesión e ingresar de nuevo.	Preferencias conservadas en la cuenta.	Aprobado / No aprobado

Nota. Elaborado por los autores a partir de los requisitos funcionales y de los manuales de usuario y administrador.

Anexo C. Plantilla de Acta de Aceptación

Tabla 17

Acta de Aceptación de la Implementación

Campo	Contenido
Sistema y versión	Sistema de Gestión de Lavado Vehicular, versión ____
Fecha y lugar	
Entorno implementado	Servidor ____, dirección de acceso ____
Casos de aceptación ejecutados y aprobados	____ de 16
Observaciones y severidad	
Decisión	Aceptación total / Aceptación condicionada / Rechazo
Compromisos y fechas (si es condicionada)	
Firma del responsable del negocio o instructora	
Firma del líder de proyecto	

Anexo D. Plantilla de Bitácora de Implementación

Tabla 18

Bitácora de Implementación

Fecha y hora	Fase y paso	Responsable	Resultado (correcto / con novedad)	Evidencia	Observaciones

Anexo E. Resumen de Comandos

```
# Fase 2: construcción y verificación
./mvnw -B verify                                  # cada servicio Java y el gateway
dotnet test                                       # lavarapido-payment-service
npx ng test --watch=false                         # Front-end-proyecto-web
node --test "e2e/*.test.js"                      # desde lavarapido-infra, con los contenedores arriba
npm ci && npx ng build                            # Front-end-proyecto-web
docker compose config --quiet && docker compose --profile app build

# Fase 3: respaldo previo (ver sección 11.4)
bash ~/lavarapido/lavarapido-infra/deploy/backup-db.sh   # servidor público (update-server.sh lo hace solo)

# Fase 4: infraestructura y base de datos
docker compose up -d && docker compose ps

# Fase 5: servicios y gateway
docker compose --profile app up -d
for port in 3001 3002 3003 3004 3005 3006 8080; do curl -s http://localhost:$port/actuator/health; echo; done

# Fase 6: aplicación web
npx ng serve --host 0.0.0.0

# Reversión (ver sección 12)
bash ~/lavarapido/lavarapido-infra/deploy/restore-db.sh <respaldo.bak.gz>   # servidor público
docker compose stop api-gateway security-service customer-service booking-service \
  operations-service payment-service notification-service
```